

--	--	--	--	--	--	--	--	--	--

Date: 26/02/2026
Duration: 1 hour 45

CONTINUOUS ASSESSMENT 1
II YEAR – SEMESTER 4
CSE23AE204 - PCP III
(B.Tech. E01,E02,E03,E05,E06)(B.Sc E61)

SET E

Question 1: Problem Statement

You are part of a **resource planning team** responsible for validating symmetric allocation grids used by internal tools.

The system represents allocations in a **uniform, slanted layout** where:

- Each row contains the **same number of allocation units**
- Alignment is achieved through **leading spaces**
- The grid must appear symmetric when rendered

To support automated validation, the grid must be returned as an **array/list of strings**, where each string represents one row.

Your task is to generate the allocation grid based on a given integer N.

Input Format

- A single integer N representing the size of the grid.

Output Format

- Return an **array/list of strings**
- Each string represents one row of the grid

Example 1

Input

4

Output

```
[
  " 1 2 3 4",
  " 1 2 3 4",
  " 1 2 3 4",
  "1 2 3 4"
]
```

Question 2: Problem Statement

Problem Statement

You are given a string consisting of lowercase English letters and a target character.

Starting from the beginning of the string, reverse the characters **up to and including the first occurrence** of the target character.

If the target character does not appear in the string, return the original string unchanged.

Input Format

- A string s
- A character ch

Output Format

- Return the resulting string after performing the controlled prefix reversal

Example 1

Input:

s = "abcdefd"

ch = "d"

Output:

dcbaefd

Explanation:

The first occurrence of 'd' is at index 3.

The prefix "abcd" is reversed to "dcba", and the remaining substring "efd" remains unchanged.

Question 3: Problem Statement

You are developing a **Smart Home Device Control System** for a modern IoT-based home.

The system processes **one device command at a time**, provided as a **space-separated string**.

Each device belongs to a specific category and performs operations differently.

The system must:

- Identify the device type from input
- Perform the device operation based on its configuration
- Use **abstraction and polymorphism** to support multiple device types
- Assign a unique device ID using a **static variable**
- Return the final computed output of the device operation

Device Operation Rules

Smart Light

- Power value represents brightness per level
- Final output is calculated based on brightness and level

Smart Fan

- Power value represents speed power
- Final output is calculated based on power and speed level

Smart AC

- Power value represents cooling capacity
- Final output is calculated based on capacity and mode level

Input Format

A **single space-separated string**.

Smart Light: LIGHT <DeviceName> <PowerValue>
<Level>

Smart Fan: FAN <DeviceName> <PowerValue> <Level>

Smart AC: AC <DeviceName> <PowerValue> <Level>

Output Format

Return an **integer value** representing the **final device operation result**.

Example 1

Input

```
{ "device": "FAN Fan1 60 3" }
```

Output

180

Explanation

The fan computes the operation result based on its power value and speed level.